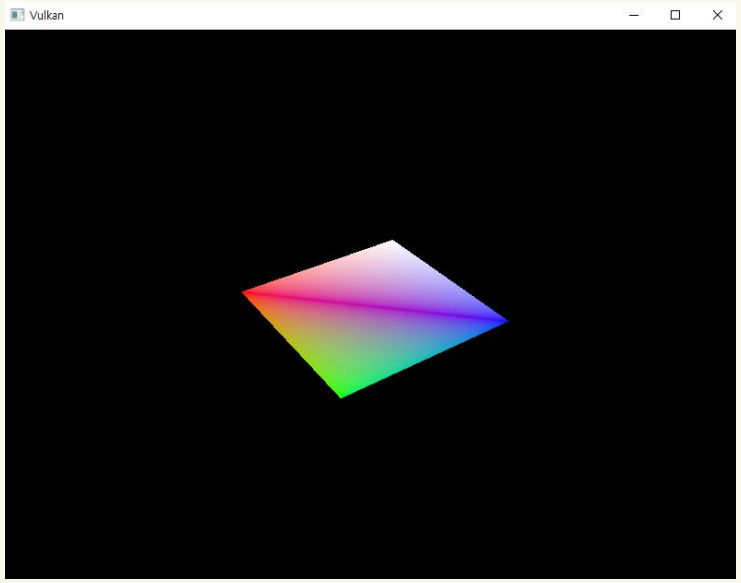
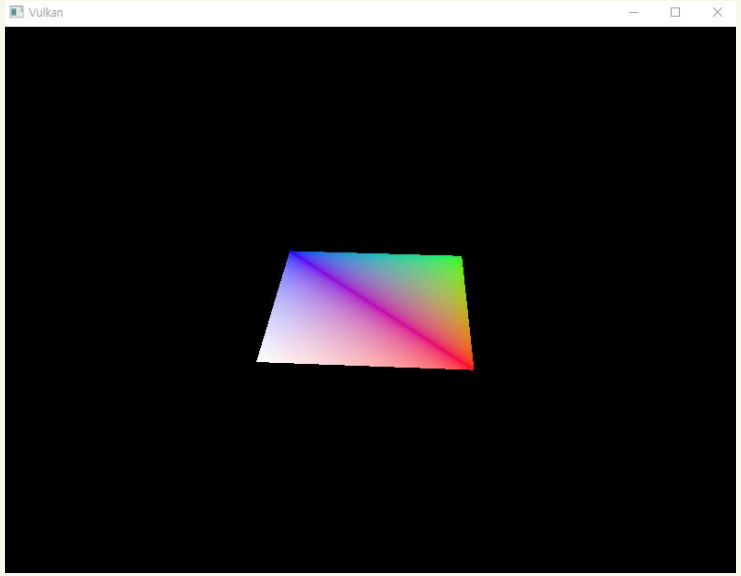
02. Managing Resources in Vulkan (Buffers)

Objectives of this Lecture

- **Learn about the functions of Vulkan's Vertex Buffer and Uniform Buffer**
  - Vertex Buffer : stores the vertex information
  - Uniform Buffer : Stores uniform data (e.g. Time, Animation progress)

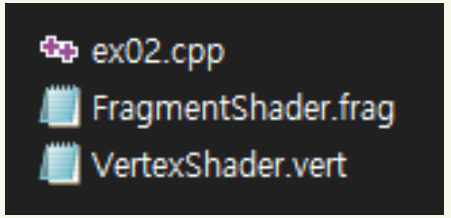
- **Create a rotating rectangle**
  - Practice assignment ‘ex02’





Practice assignment

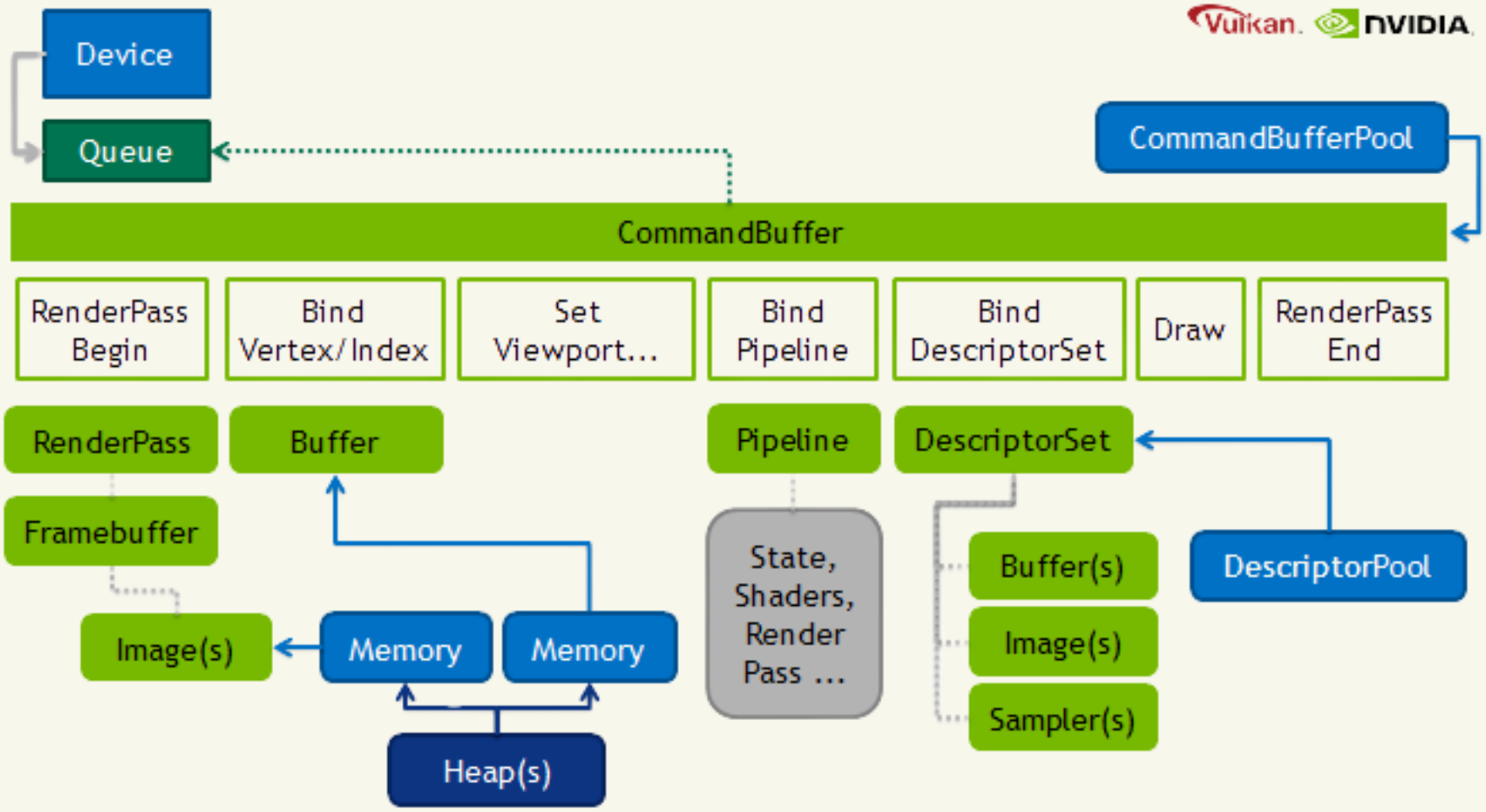
- **Create a rotating rectangle**
  - Check file name before compiling shader



- **Check Vertex buffer structure**
  - Stores vertex data on the GPU and provides it to the graphics pipeline

- **Check Uniform buffer structure**
  - Stores uniform data on the GPU and provides it to shaders (Adjusting global properties of objects)

Vulkan Graphics Workflow Overview



Vertex Buffers

- **Vertex Input**
- **GLM :** Provides linear algebra-related types such as vectors and matrices

```
struct Vertex {
    glm::vec2 pos;
    glm::vec3 color;
};
const std::vector<Vertex> vertices = {
    {{0.0f, -0.5f}, {1.0f, 0.0f, 0.0f}},
    {{0.5f, 0.5f}, {0.0f, 1.0f, 0.0f}},
    {{-0.5f, 0.5f}, {0.0f, 0.0f, 1.0f}}
};
```

Vertex Buffers

- **In variable Binding**
- **To instruct Vulkan on how to pass data to the vertex shader** : Two structures are required.
  - VkVertexInputBindingDescription
  - VkVertexInputAttributeDescription

```
static VkVertexInputBindingDescription getBindingDescription() {
    VkVertexInputBindingDescription bindingDescription{};
    bindingDescription.binding = 0;
    bindingDescription.stride = sizeof(Vertex);
    bindingDescription.inputRate = VK_VERTEX_INPUT_RATE_VERTEX;

    return bindingDescription;
}
```

- **VkVertexInputBindingDescription** : Structure specifying vertex input binding description
  - **binding** : The binding number that this structure describes.
  - **stride** : The byte stride between consecutive elements within the buffer.
  - **inputRate** : A VkVertexInputRate value specifying whether vertex attribute addressing is a function of the vertex index or of the instance index.

Vertex Buffers

- **In variable Binding**

```
    static std::array<VkVertexInputAttributeDescription, 2> getAttributeDescriptions() {
        std::array<VkVertexInputAttributeDescription, 2> attributeDescriptions{};

        attributeDescriptions[0].binding = 0;
        attributeDescriptions[0].location = 0;
        attributeDescriptions[0].format = VK_FORMAT_R32G32_SFLOAT;
        attributeDescriptions[0].offset = offsetof(Vertex, pos);

        attributeDescriptions[1].binding = 0;
        attributeDescriptions[1].location = 1;
        attributeDescriptions[1].format = VK_FORMAT_R32G32B32_SFLOAT;
        attributeDescriptions[1].offset = offsetof(Vertex, color);

        return attributeDescriptions;
    }
};
```

- **float:** VK_FORMAT_R32_SFLOAT
- **vec2:** VK_FORMAT_R32G32_SFLOAT
- **vec3:** VK_FORMAT_R32G32B32_SFLOAT
- **vec4:** VK_FORMAT_R32G32B32A32_SFLOAT
- **ivec2:** VK_FORMAT_R32G32_SINT, a 2-component vector of 32-bit signed integers
- **uvec4:** VK_FORMAT_R32G32B32A32_UINT, a 4-component vector of 32-bit unsigned integers
- **double:** VK_FORMAT_R64_SFLOAT, a double-precision (64-bit) float

- **VkVertexInputAttributeDescription** : Structure specifying vertex input attribute description
  - **location** : The shader input location number for this attribute.
  - **binding** : The binding number which this attribute takes its data from.
  - **format** : The size and type of the vertex attribute data.
  - **offset** : A byte offset of this attribute relative to the start of an element in the vertex input binding.

```
# version 450
 layout(location = 0) in dvec3 inPosition;
 layout(location = 2) in vec3 inColor;
```

Vertex Buffers

- **Set up the graphics pipeline**

```
VkPipelineVertexInputStateCreateInfo vertexInputInfo{};
vertexInputInfo.sType = VK_STRUCTURE_TYPE_PIPELINE_VERTEX_INPUT_STATE_CREATE_INFO;
```

```
vertexInputInfo.vertexBindingDescriptionCount = 1;
vertexInputInfo.vertexAttributeDescriptionCount = static_cast<uint32_t>(attributeDescriptions.size());
vertexInputInfo.pVertexBindingDescriptions = &bindingDescription;
vertexInputInfo.pVertexAttributeDescriptions = attributeDescriptions.data();
```

- **VkPipelineVertexInputStateCreateInfo** : Structure specifying parameters of a newly created pipeline vertex input state
  - **vertexBindingDescriptionCount** : The number of vertex binding descriptions provided in pVertexBindingDescriptions.
  - **pVertexBindingDescriptions** : A pointer to an array of VkVertexInputBindingDescription structures.
  - **vertexAttributeDescriptionCount** : The number of vertex attribute descriptions provided in pVertexAttributeDescriptions.
  - **pVertexAttributeDescriptions** : A pointer to an array of VkVertexInputAttributeDescription structures.

Vertex Buffers

- **Vulkan buffers store data, but don't self-allocate memory**

- **Creating Vertex Buffer**

```
void createVertexBuffer() {
    VkBufferCreateInfo bufferInfo{};
    bufferInfo.sType = VK_STRUCTURE_TYPE_BUFFER_CREATE_INFO;
    bufferInfo.size = sizeof(vertices[0]) * vertices.size();
    bufferInfo.usage = VK_BUFFER_USAGE_VERTEX_BUFFER_BIT;
    bufferInfo.sharingMode = VK_SHARING_MODE_EXCLUSIVE;

    if (vkCreateBuffer(device, &bufferInfo, nullptr, &vertexBuffer) != VK_SUCCESS) {
        throw std::runtime_error("failed to create vertex buffer!");
    }
```

- **VkBufferCreateInfo** : Structure specifying the parameters of a newly created buffer object
    - **size** : A size in bytes of the buffer to be created.
    - **usage** : A bitmask of VkBufferUsageFlagBits specifying allowed usages of the buffer.
    - **sharingMode** : A VkSharingMode value specifying the sharing mode of the buffer when it will be accessed by multiple queue families.

Vertex Buffers

- **Memory Assignment**

```
VkMemoryRequirements memRequirements;
vkGetBufferMemoryRequirements(device, vertexBuffer, &memRequirements);
```

- **VkMemoryRequirements fields:**
  - **size:** The size of the required amount of memory in bytes, may differ from bufferInfo.size.
  - **alignment**: The offset in bytes where the buffer begins in the allocated region of memory, depends on bufferInfo.usage and bufferInfo.flags.
  - **memoryTypeBits:** Bit field of the memory types that are suitable for the buffer.

Vertex Buffers

- **Find Memory type**

```
VkPhysicalDeviceMemoryProperties memProperties;
vkGetPhysicalDeviceMemoryProperties(physicalDevice, &memProperties);

    for (uint32_t i = 0; i < memProperties.memoryTypeCount; i++) {
        if ((typeFilter & (1 << i)) && (memProperties.memoryTypes[i].propertyFlags & properties) == properties) {
            return i;
        }
    }

    throw std::runtime_error("failed to find suitable memory type!");
}
```

- **VkPhysicalDeviceMemoryProperties** : Structure specifying physical device memory properties
  - **memoryTypeCount** : The number of valid elements in the memoryTypes array.
  - **memoryTypes** : An array of VK_MAX_MEMORY_TYPES VkMemoryType structures describing the memory types that can be used to access memory allocated from the heaps specified by memoryHeaps.
  - **memoryHeapCount** : The number of valid elements in the memoryHeaps array.
  - **memoryHeaps** : An array of VK_MAX_MEMORY_HEAPS VkMemoryHeap structures describing the memory heaps from which memory can be allocated.

Vertex Buffers

- **Find Memory type**

```
VkPhysicalDeviceMemoryProperties memProperties;
vkGetPhysicalDeviceMemoryProperties(physicalDevice, &memProperties);

    for (uint32_t i = 0; i < memProperties.memoryTypeCount; i++) {
        if ((typeFilter & (1 << i)) && (memProperties.memoryTypes[i].propertyFlags & properties) == properties) {
            return i;
        }
    }

    throw std::runtime_error("failed to find suitable memory type!");
}
```

- **vkGetPhysicalDeviceMemoryProperties** : Reports memory information for the specified physical device
  - **memoryTypeCount** : The number of valid elements in the memoryTypes array.
  - **physicalDevice** : The handle to the device to query.
  - **pMemoryProperties :** A pointer to a VkPhysicalDeviceMemoryProperties structure in which the properties are returned.

Vertex Buffers

- **Memory Allocation**

```
VkMemoryAllocateInfo allocInfo{};
allocInfo.sType = VK_STRUCTURE_TYPE_MEMORY_ALLOCATE_INFO;
allocInfo.allocationSize = memRequirements.size;
allocInfo.memoryTypeIndex = findMemoryType(memRequirements.memoryTypeBits, VK_MEMORY_PROPERTY_HOST_VISIBLE_BIT |
VK_MEMORY_PROPERTY_HOST_COHERENT_BIT);
```

- **VkMemoryAllocateInfo** : Structure containing parameters of a memory allocation
  - **pNext** : NULL or a pointer to a structure extending this structure.
  - **allocationSize** : The size of the allocation in bytes.
  - **memoryTypeIndex** : An index identifying a memory type from the memoryTypes array of the VkPhysicalDeviceMemoryProperties structure.

```
VkBuffer vertexBuffer;
VkDeviceMemory vertexBufferMemory;

...

 if (vkAllocateMemory(device, &allocInfo, nullptr, &vertexBufferMemory) != VK_SUCCESS) {
        throw std::runtime_error("failed to allocate vertex buffer memory!");
    }

    vkBindBufferMemory(device, vertexBuffer, vertexBufferMemory, 0);
```

- **VkBuffer** : Opaque handle to a buffer object
- **VkDeviceMemory** : Opaque handle to a device memory object

Vertex Buffers

- **Filling the Buffer**

```
void* data;
vkMapMemory(device, vertexBufferMemory, 0, bufferInfo.size, 0, &data);
memcpy(data, vertices.data(), (size_t)bufferInfo.size);
vkUnmapMemory(device, vertexBufferMemory);
```

- **vkMapMemory** : Map a memory object into application address space
  - **device** : The logical device that owns the memory.
  - **memory** : The VkDeviceMemory object to be mapped.
  - **offset** : A zero-based byte offset from the beginning of the memory object.
  - **size** : The size of the memory range to map, or VK_WHOLE_SIZE to map from offset to the end of the allocation.
  - **flags** : reserved for future use.
  - **ppData** : A pointer to a void* variable in which a host-accessible pointer to the beginning of the mapped range is returned. This pointer minus offset must be aligned to at least VkPhysicalDeviceLimits::minMemoryMapAlignment.

- **vkUnmapMemory** : Unmap a previously mapped memory object
  - **device** : The logical device that owns the memory.
  - **memory** : The memory object to be unmapped.

Vertex Buffers

- **Command**

```
vkCmdBindPipeline(commandBuffer, VK_PIPELINE_BIND_POINT_GRAPHICS, graphicsPipeline);

VkBuffer vertexBuffers[] = {vertexBuffer};
VkDeviceSize offsets[] = {0};
vkCmdBindVertexBuffers(commandBuffer, 0, 1, vertexBuffers, offsets);

vkCmdDraw(commandBuffer, static_cast<uint32_t>(vertices.size()), 1, 0, 0);
```

- **vkCmdBindPipeline** : Bind a pipeline object to a command buffer
  - **commandBuffer** : The command buffer that the pipeline will be bound to.
  - **pipelineBindPoint** : A VkPipelineBindPoint value specifying to which bind point the pipeline is bound. Binding one does not disturb the others.
  - **pipeline** : The pipeline to be bound.
- **vkCmdBindVertexBuffers** : Bind vertex buffers to a command buffer
  - **commandBuffer** : The command buffer into which the command is recorded.
  - **firstBinding** : The index of the first vertex input binding whose state is updated by the command.
  - **bindingCount** : The number of vertex input bindings whose state is updated by the command.
  - **pBuffers** : A pointer to an array of buffer handles.
  - **pOffsets** : A pointer to an array of buffer offsets.
- **vkCmdDraw** : Draw primitives
  - **vertexCount** : The number of vertices to draw.
  - **instanceCount** : The number of instances to draw.
  - **firstVertex** : The index of the first vertex to draw.
  - **firstInstance** : The instance ID of the first instance to draw.

Vertex Buffers

Efficient Memory

- **CPU Accessible Memory vs. GPU Local Memory**
- **Use of two buffers**
  - Staging buffer → local buffer
- **Buffer copy**
  - Transfer queue

```
void createVertexBuffer() {
    VkDeviceSize bufferSize = sizeof(vertices[0]) * vertices.size();

    VkBuffer stagingBuffer;
    VkDeviceMemory stagingBufferMemory;
    createBuffer(bufferSize, VK_BUFFER_USAGE_TRANSFER_SRC_BIT, VK_MEMORY_PROPERTY_HOST_VISIBLE_BIT |
VK_MEMORY_PROPERTY_HOST_COHERENT_BIT, stagingBuffer, stagingBufferMemory);

    void* data;
    vkMapMemory(device, stagingBufferMemory, 0, bufferSize, 0, &data);
    memcpy(data, vertices.data(), (size_t)bufferSize);
    vkUnmapMemory(device, stagingBufferMemory);

    createBuffer(bufferSize, VK_BUFFER_USAGE_TRANSFER_DST_BIT | VK_BUFFER_USAGE_VERTEX_BUFFER_BIT, VK_MEMORY_PROPERTY_DEVICE_LOCAL_BIT,
vertexBuffer, vertexBufferMemory);
```

Vertex Buffers

```
void copyBuffer(VkBuffer srcBuffer, VkBuffer dstBuffer, VkDeviceSize size) {
    VkCommandBufferAllocateInfo allocInfo{};
    allocInfo.sType = VK_STRUCTURE_TYPE_COMMAND_BUFFER_ALLOCATE_INFO;
    allocInfo.level = VK_COMMAND_BUFFER_LEVEL_PRIMARY;
    allocInfo.commandPool = commandPool;
    allocInfo.commandBufferCount = 1;

    VkCommandBuffer commandBuffer;
    vkAllocateCommandBuffers(device, &allocInfo, &commandBuffer);

    VkCommandBufferBeginInfo beginInfo{};
    beginInfo.sType = VK_STRUCTURE_TYPE_COMMAND_BUFFER_BEGIN_INFO;
    beginInfo.flags = VK_COMMAND_BUFFER_USAGE_ONE_TIME_SUBMIT_BIT;

    vkBeginCommandBuffer(commandBuffer, &beginInfo);

    VkBufferCopy copyRegion{};
    copyRegion.size = size;
    vkCmdCopyBuffer(commandBuffer, srcBuffer, dstBuffer, 1, &copyRegion);

    vkEndCommandBuffer(commandBuffer);

    VkSubmitInfo submitInfo{};
    submitInfo.sType = VK_STRUCTURE_TYPE_SUBMIT_INFO;
    submitInfo.commandBufferCount = 1;
    submitInfo.pCommandBuffers = &commandBuffer;

    vkQueueSubmit(graphicsQueue, 1, &submitInfo, VK_NULL_HANDLE);
    vkQueueWaitIdle(graphicsQueue);

    vkFreeCommandBuffers(device, commandPool, 1, &commandBuffer);
}
```

- **VkCommandBufferAllocateInfo** : Structure specifying the allocation parameters for command buffer object
- **VkCommandBuffer** : Opaque handle to a command buffer object
- **VkCommandBufferBeginInfo** : Structure specifying a command buffer begin operation
- **VkBufferCopy** : Structure specifying a buffer copy operation
- **VkSubmitInfo** : Structure specifying a queue submit operation
- **vkQueueSubmit** : Submits a sequence of semaphores or command buffers to a queue
- **vkQueueWaitIdle** : Wait for a queue to become idle
- **vkFreeCommandBuffers** : Free command buffers

Uniform Buffers

Uniform Variables in Vulkan

- **Just as a resource**

- **Resource descriptors**
  - Descriptor Layout for pipeline
  - Allocate a descriptor set
  - Bind

- **Uniform Variables → Uniform Buffer Object**

Uniform Buffers

- **Uniform value example**

```
struct UniformBufferObject {
    glm::mat4 model;
    glm::mat4 view;
    glm::mat4 proj;
};
```

- **Shader code example**

```
#version 450

    layout(binding = 0) uniform UniformBufferObject {
    mat4 model;
    mat4 view;
    mat4 proj;
} ubo;

layout(location = 0) in vec2 inPosition;
layout(location = 1) in vec3 inColor;

layout(location = 0) out vec3 fragColor;

void main() {
    gl_Position = ubo.proj * ubo.view * ubo.model * vec4(inPosition, 0.0, 1.0);
    fragColor = inColor;
}
```

Note: location vs. binding

Uniform Buffers

- **Descriptor Layout**

```
VkDescriptorSetLayoutBinding uboLayoutBinding{};
uboLayoutBinding.binding = 0;
uboLayoutBinding.descriptorCount = 1;
uboLayoutBinding.descriptorType = VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER;
uboLayoutBinding.pImmutableSamplers = nullptr;
uboLayoutBinding.stageFlags = VK_SHADER_STAGE_VERTEX_BIT;
```

- **VkDescriptorSetLayoutBinding** : Structure specifying a descriptor set layout binding
  - **binding** : The binding number of this entry and corresponds to a resource of the same binding number in the shader stages.
  - **descriptorType** : A VkDescriptorType specifying which type of resource descriptors are used for this binding.
  - **descriptorCount** : The number of descriptors contained in the binding, accessed in a shader as an array, except if descriptorType is VK_DESCRIPTOR_TYPE_INLINE_UNIFORM_BLOCK in which case descriptorCount is the size in bytes of the inline uniform block.
  - **stageFlags** member : A bitmask of VkShaderStageFlagBits specifying which pipeline shader stages can access a resource for this binding.
  - **pImmutableSamplers** affects initialization of samplers. If descriptorType specifies a VK_DESCRIPTOR_TYPE_SAMPLER or VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER type descriptor, then pImmutableSamplers can be used to initialize a set of immutable samplers.

Uniform Buffers

- **Descriptor Layout**

```
VkDescriptorSetLayout descriptorSetLayout;
VkPipelineLayout pipelineLayout;
```

```
if (vkCreateDescriptorSetLayout(device, &layoutInfo, nullptr, &descriptorSetLayout) != VK_SUCCESS) {
        throw std::runtime_error("failed to create descriptor set layout!");
    }
```

- **VkDescriptorSetLayout** : Opaque handle to a descriptor set layout object
- **VkPipelineLayout** : Opaque handle to a pipeline layout object
- **vkCreateDescriptorSetLayout** : Create a new descriptor set layout
  - **device** : The logical device that creates the descriptor set layout.
  - **pCreateInfo** : A pointer to a VkDescriptorSetLayoutCreateInfo structure specifying the state of the descriptor set layout object.
  - **pAllocator** controls host memory allocation as described in the Memory Allocation chapter.
  - **pSetLayout** : A pointer to a VkDescriptorSetLayout handle in which the resulting descriptor set layout object is returned.

Uniform Buffers

- **Buffer Allocation**

```
std::vector<VkBuffer> uniformBuffers;
std::vector<VkDeviceMemory> uniformBuffersMemory;
std::vector<void*> uniformBuffersMapped;
```

Note: To make the pipeline doesn't stop when uniform value changes

```
void createUniformBuffers() {
    VkDeviceSize bufferSize = sizeof(UniformBufferObject);

    uniformBuffers.resize(MAX_FRAMES_IN_FLIGHT);
    uniformBuffersMemory.resize(MAX_FRAMES_IN_FLIGHT);
    uniformBuffersMapped.resize(MAX_FRAMES_IN_FLIGHT);

    for (size_t i = 0; i < MAX_FRAMES_IN_FLIGHT; i++) {
        createBuffer(bufferSize, VK_BUFFER_USAGE_UNIFORM_BUFFER_BIT, VK_MEMORY_PROPERTY_HOST_VISIBLE_BIT |
VK_MEMORY_PROPERTY_HOST_COHERENT_BIT, uniformBuffers[i], uniformBuffersMemory[i]);

        vkMapMemory(device, uniformBuffersMemory[i], 0, bufferSize, 0, &uniformBuffersMapped[i]);
    }
}
```

- **VkDeviceSize** : Vulkan device memory size and offsets

Uniform Buffers

Buffer Copy

- **Current frame**
  - currentFram = (currentFrame + 1) % MAX_FRAMES_IN_FLIGHT;

```
void updateUniformBuffer(uint32_t currentImage) {
    static auto startTime = std::chrono::high_resolution_clock::now();

    auto currentTime = std::chrono::high_resolution_clock::now();
    float time = std::chrono::duration<float, std::chrono::seconds::period>(currentTime - startTime).count();

    UniformBufferObject ubo{};
    ubo.model = glm::rotate(glm::mat4(1.0f), time * glm::radians(90.0f), glm::vec3(0.0f, 0.0f, 1.0f));
    ubo.view = glm::lookAt(glm::vec3(2.0f, 2.0f, 2.0f), glm::vec3(0.0f, 0.0f, 0.0f), glm::vec3(0.0f, 0.0f, 1.0f));
    ubo.proj = glm::perspective(glm::radians(45.0f), swapChainExtent.width / (float)swapChainExtent.height, 0.1f, 10.0f);
    ubo.proj[1][1] *= -1;

    memcpy(uniformBuffersMapped[currentImage], &ubo, sizeof(ubo));
}
```

- **updateUniformBuffer** : Calculates time elapsed since rendering started

Uniform Buffers

- **Bind from Descriptor Pool**
  - **Descriptor Pool :** Creates a memory pool for descriptors of uniform buffer type.

```
VkDescriptorPoolSize poolSize{};
poolSize.type = VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER;
poolSize.descriptorCount = static_cast<uint32_t>(MAX_FRAMES_IN_FLIGHT);

VkDescriptorPoolCreateInfo poolInfo{};
poolInfo.sType = VK_STRUCTURE_TYPE_DESCRIPTOR_POOL_CREATE_INFO;
poolInfo.poolSizeCount = 1;
poolInfo.pPoolSizes = &poolSize;
poolInfo.maxSets = static_cast<uint32_t>(MAX_FRAMES_IN_FLIGHT);

if (vkCreateDescriptorPool(device, &poolInfo, nullptr, &descriptorPool) != VK_SUCCESS) {
    throw std::runtime_error("failed to create descriptor pool!");
}
```

- **VkDescriptorPoolSize** : Structure specifying descriptor pool size
  - **type** : The type of descriptor.
  - **descriptorCount** : The number of descriptors of that type to allocate. If type is VK_DESCRIPTOR_TYPE_INLINE_UNIFORM_BLOCK then descriptorCount is the number of bytes to allocate for descriptors of this type.

- **VkDescriptorPoolCreateInfo** : Structure specifying parameters of a newly created descriptor pool
  - **maxSets** : The maximum number of descriptor sets that can be allocated from the pool.
  - **poolSizeCount** : The number of elements in pPoolSizes.
  - **pPoolSizes** : A pointer to an array of VkDescriptorPoolSize structures, each containing a descriptor type and number of descriptors of that type to be allocated in the pool.

Uniform Buffers

- **Bind from Descriptor Pool**
  - **Descriptor Sets :** Defines a set of descriptors allocated from the memory pool.

```
std::vector<VkDescriptorSetLayout> layouts(MAX_FRAMES_IN_FLIGHT, descriptorSetLayout);
 VkDescriptorSetAllocateInfo allocInfo{};
 allocInfo.sType = VK_STRUCTURE_TYPE_DESCRIPTOR_SET_ALLOCATE_INFO;
 allocInfo.descriptorPool = descriptorPool;
 allocInfo.descriptorSetCount = static_cast<uint32_t>(MAX_FRAMES_IN_FLIGHT);
 allocInfo.pSetLayouts = layouts.data();

 descriptorSets.resize(MAX_FRAMES_IN_FLIGHT);
 if (vkAllocateDescriptorSets(device, &allocInfo, descriptorSets.data()) != VK_SUCCESS) {
    throw std::runtime_error("failed to allocate descriptor sets!");
 }
```

- **VkDescriptorSetAllocateInfo** : Structure specifying the allocation parameters for descriptor sets
  - **descriptorPool** : The pool which the sets will be allocated from.
  - **descriptorSetCount** determines the number of descriptor sets to be allocated from the pool.
  - **pSetLayouts** : A pointer to an array of descriptor set layouts, with each member specifying how the corresponding descriptor set is allocated.

Uniform Buffers

- **Descriptor Set up**

```
for (size_t i = 0; i < MAX_FRAMES_IN_FLIGHT; i++) {
VkDescriptorBufferInfo bufferInfo{};
bufferInfo.buffer = uniformBuffers[i];
bufferInfo.offset = 0;
bufferInfo.range = sizeof(UniformBufferObject);
```

- **VkDescriptorBufferInfo** : Structure specifying descriptor buffer information
  - **buffer** : VK_NULL_HANDLE or the buffer resource.
  - **offset** : The offset in bytes from the start of buffer. Access to buffer memory via this descriptor uses addressing that is relative to this starting offset.
  - **range** : The size in bytes that is used for this descriptor update, or VK_WHOLE_SIZE to use the range from offset to the end of the buffer.

Uniform Buffers

- **Descriptor Set up**

```
VkWriteDescriptorSet descriptorWrite{};
descriptorWrite.sType = VK_STRUCTURE_TYPE_WRITE_DESCRIPTOR_SET;
descriptorWrite.dstSet = descriptorSets[i];
descriptorWrite.dstBinding = 0;
descriptorWrite.dstArrayElement = 0;
```

```
descriptorWrite.descriptorType = VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER;
descriptorWrite.descriptorCount = 1;
descriptorWrite.pBufferInfo = &bufferInfo;
descriptorWrite.pImageInfo = nullptr;  // optional
descriptorWrite.pTexelBufferView = nullptr; // optional
vkUpdateDescriptorSets(device, 1, &descriptorWrite, 0, nullptr);
```

- **VkWriteDescriptorSet** : Structure specifying the parameters of a descriptor set write operation
  - **dstSet** : The destination descriptor set to update.
  - **dstBinding** : The descriptor binding within that set.
  - **dstArrayElement** : The starting element in that array.
  - **descriptorCount** : The number of descriptors to update.
    - the number of elements in pImageInfo
    - the number of elements in pBufferInfo
    - the number of elements in pTexelBufferView
    - a value matching the dataSize member of a VkWriteDescriptorSetInlineUniformBlock structure in the pNext chain
    - a value matching the accelerationStructureCount of a VkWriteDescriptorSetAccelerationStructureKHR structure in the pNext chain
  - **descriptorType** : A VkDescriptorType specifying the type of each descriptor in pImageInfo, pBufferInfo, or pTexelBufferView, as described below.
  - **pImageInfo** : A pointer to an array of VkDescriptorImageInfo structures or is ignored, as described below.
  - **pBufferInfo** : A pointer to an array of VkDescriptorBufferInfo structures or is ignored, as described below.
  - **pTexelBufferView** : A pointer to an array of VkBufferView handles as described in the Buffer Views section or is ignored, as described below.

Uniform Buffers

- **Using Descriptor Sets**

```
vkCmdBindDescriptorSets(commandBuffer, VK_PIPELINE_BIND_POINT_GRAPHICS, pipelineLayout, 0, 1, &descriptorSets[currentFrame], 0, nullptr);

vkCmdDrawIndexed(commandBuffer, static_cast<uint32_t>(indices.size()), 1, 0, 0, 0);
```

- **vkCmdBindDescriptorSets**: Binds descriptor sets to a command buffer
  - **commandBuffer** : The command buffer that the descriptor sets will be bound to.
  - **pipelineBindPoint** : A VkPipelineBindPoint indicating the type of the pipeline that will use the descriptors.
  - **layout** : A VkPipelineLayout object used to program the bindings.
  - **firstSet** : The set number of the first descriptor set to be bound.
  - **descriptorSetCount** : The number of elements in the pDescriptorSets array.
  - **pDescriptorSets** : A pointer to an array of handles to VkDescriptorSet objects describing the descriptor sets to bind to.
  - **dynamicOffsetCount** : The number of dynamic offsets in the pDynamicOffsets array.
  - **pDynamicOffsets** : A pointer to an array of uint32_t values specifying dynamic offsets.

Uniform Buffers

- **Using Descriptor Sets**

```
vkCmdBindDescriptorSets(commandBuffer, VK_PIPELINE_BIND_POINT_GRAPHICS, pipelineLayout, 0, 1, &descriptorSets[currentFrame], 0, nullptr);

vkCmdDrawIndexed(commandBuffer, static_cast<uint32_t>(indices.size()), 1, 0, 0, 0);
```

- **vkCmdDrawIndexed**: Draw primitives with indexed vertices
  - **commandBuffer** : The command buffer into which the command is recorded.
  - **indexCount** : The number of vertices to draw.
  - **instanceCount** : The number of instances to draw.
  - **firstIndex** : The base index within the index buffer.
  - **vertexOffset** : The value added to the vertex index before indexing into the vertex buffer.
  - **firstInstance** : The instance ID of the first instance to draw.

To be continued

Textures and Other Resources

- Texture Mapping
- Depth Buffering